MILESTONE 6 &5

CROP GROWTH SIMULATION

```
// milestone5_fixed.cpp
#include <iostream>
#include <vector>
#include <string>
#include <cmath>
#include <thread>
#include <mutex>
#include <atomic>
#include <algorithm>
#include <numeric>
#include <chrono>
#include <sstream>
#include <iomanip>
#include <memory>

using namespace std;

// ------------------------------ Enumerations ------------------------------
enum class CropStage { SEEDLING, VEGETATIVE, REPRODUCTIVE, MATURING, READY,
HARVESTED };
enum class IrrigationState { NONE, LOW, MODERATE, HIGH, CRITICAL };

string stageStr(CropStage s) {
    switch(s) {
```

```
        case CropStage::SEEDLING: return "Seedling";
        case CropStage::VEGETATIVE: return "Vegetative";
        case CropStage::REPRODUCTIVE: return "Reproductive";
        case CropStage::MATURING: return "Maturing";
        case CropStage::READY: return "Harvest Ready";
        case CropStage::HARVESTED: return "Harvested";
        default: return "?";
    }
}

string irriStr(IrrigationState s) {
    switch(s) {
        case IrrigationState::NONE: return "None";
        case IrrigationState::LOW: return "Low";
        case IrrigationState::MODERATE: return "Moderate";
        case IrrigationState::HIGH: return "High";
        case IrrigationState::CRITICAL: return "Critical";
        default: return "?";
    }
}

// ------------------------------ Template (Generics) ------------------------------
template<typename T>
T computeAverage(const vector<T>& data) {
    if (data.empty()) return T(0);
    T sum = accumulate(data.begin(), data.end(), T(0));
```

```
    return sum / data.size();
}

// ------------------------------ Field (non-copyable, movable) ------------------------------
class Field {
    string name;
    double moisture;
    mutable mutex mtx;
public:
    Field(const string& n) : name(n), moisture(50.0) {}
    Field(const Field&) = delete;
    Field& operator=(const Field&) = delete;
    Field(Field&&) = default;
    Field& operator=(Field&&) = default;

    void update(double rain, double temp) {
        lock_guard<mutex> lock(mtx);
        double evap = 2.0 + temp/15.0;
        moisture += rain*0.8 - evap;
        if (moisture < 0) moisture = 0;
        if (moisture > 100) moisture = 100;
    }
    void irrigate(double amount) {
        lock_guard<mutex> lock(mtx);
        moisture += amount / 100.0;
        if (moisture > 100) moisture = 100;
```

```
    }
    double getMoisture() const {
        lock_guard<mutex> lock(mtx);
        return moisture;
    }
    IrrigationState need() const {
        double m = getMoisture();
        if (m >= 70) return IrrigationState::NONE;
        if (m >= 50) return IrrigationState::LOW;
        if (m >= 30) return IrrigationState::MODERATE;
        if (m >= 15) return IrrigationState::HIGH;
        return IrrigationState::CRITICAL;
    }
    string getName() const { return name; }
};

// ------------------------------ Crop (non-copyable) ------------------------------
class Crop {
    string name;
    double growth;
    double health;
    double waterReq, tempOpt, maxYield, baseRate;
    bool harvested;
    mutable mutex mtx;
public:
    Crop(const string& n, double wr, double topt, double myield, double br)
```

```
    : name(n), growth(0.0), health(1.0), waterReq(wr), tempOpt(topt),
      maxYield(myield), baseRate(br), harvested(false) {}
Crop(const Crop&) = delete;
Crop& operator=(const Crop&) = delete;
Crop(Crop&&) = default;

void update(double temp, double waterSupply) {
    if (harvested) return;
    double waterStress = (waterSupply <= 0) ? 0 : min(1.0, waterSupply / waterReq);
    double tempStress = (abs(temp - tempOpt) > 20) ? 0 : 1.0 - abs(temp - tempOpt)/20.0;
    double rate = baseRate * waterStress * tempStress;
    rate *= (1.0 - growth * 0.5);
    if (rate < 0) rate = 0;
    if (rate > 0.05) rate = 0.05;
    lock_guard<mutex> lock(mtx);
    growth += rate;
    if (growth > 1.0) growth = 1.0;
    health = health * 0.99 + (waterStress * tempStress) * 0.01;
}
double getGrowth() const {
    lock_guard<mutex> lock(mtx);
    return growth;
}
double getHealth() const {
    lock_guard<mutex> lock(mtx);
    return health;
```

```
    }
    CropStage stage() const {
        lock_guard<mutex> lock(mtx);
        if (harvested) return CropStage::HARVESTED;
        if (growth >= 0.9) return CropStage::READY;
        if (growth >= 0.75) return CropStage::MATURING;
        if (growth >= 0.5) return CropStage::REPRODUCTIVE;
        if (growth >= 0.25) return CropStage::VEGETATIVE;
        return CropStage::SEEDLING;
    }
    string getName() const { return name; }
    double harvest() {
        lock_guard<mutex> lock(mtx);
        if (harvested || growth < 0.9) return 0;
        double yield = maxYield * growth * health;
        harvested = true;
        return yield;
    }
};

// ------------------------------ Simulation Engine (pointers to avoid copy) ------------------------------
class SimulationEngine {
    vector<unique_ptr<Field>> fields;
    vector<unique_ptr<Crop>> crops;
    atomic<int> currentDay{0};
    atomic<bool> running{false};
```

```
thread simThread;


void simulateOneDay() {
    currentDay++;
    double temp = 20.0 + 5.0 * sin(currentDay * 0.1);
    double rain = (currentDay % 4 == 0) ? 5.0 : 0.0;


    vector<thread> workers;
    for (auto& f : fields) {
        workers.emplace_back([&f, rain, temp]() {
            f->update(rain, temp);
            if (f->need() == IrrigationState::HIGH || f->need() == IrrigationState::CRITICAL)
                f->irrigate(2000);
        });
    }
    double avgMoisture = 50.0;
    if (!fields.empty()) {
        double sum = 0;
        for (auto& f : fields) sum += f->getMoisture();
        avgMoisture = sum / fields.size();
    }
    for (auto& c : crops) {
        workers.emplace_back([&c, temp, avgMoisture]() {
            c->update(temp, avgMoisture);
        });
    }
```

```
        for (auto& t : workers) t.join();
    }

    void runLoop() {
        while (running) {
            simulateOneDay();
            this_thread::sleep_for(chrono::milliseconds(200));
        }
    }

public:
    SimulationEngine() {
        fields.push_back(make_unique<Field>("North Field"));
        fields.push_back(make_unique<Field>("South Field"));
        crops.push_back(make_unique<Crop>("Wheat", 4.5, 20.0, 6.0, 0.015));
        crops.push_back(make_unique<Crop>("Corn", 6.0, 25.0, 12.0, 0.018));
        crops.push_back(make_unique<Crop>("Tomato", 3.5, 22.0, 40.0, 0.025));
    }

    void start() {
        if (running) return;
        running = true;
        simThread = thread(&SimulationEngine::runLoop, this);
    }

    void stop() {
```

```
    running = false;
    if (simThread.joinable()) simThread.join();
}

void step() {
    simulateOneDay();
}

void showStatus() const {
    cout << "\n========== Day " << currentDay.load() << " ==========\n";
    cout << "--- Fields ---\n";
    for (auto& f : fields) {
        cout << "  " << f->getName() << ": moisture = " << fixed << setprecision(1)
             << f->getMoisture() << "%  | Need: " << irriStr(f->need()) << endl;
    }
    cout << "--- Crops ---\n";
    for (auto& c : crops) {
        cout << "  " << c->getName() << ": growth = " << (c->getGrowth() * 100)
             << "%  | stage = " << stageStr(c->stage()) << endl;
    }
}

void harvestAll() {
    cout << "\n--- Harvest Results ---\n";
    for (auto& c : crops) {
        double yield = c->harvest();
```

```
        if (yield > 0)
            cout << "  Harvested " << c->getName() << " -> " << yield << " t/ha\n";
        else
            cout << "  " << c->getName() << " not ready.\n";
    }
}

void showAverageGrowth() {
    vector<double> growths;
    for (auto& c : crops) growths.push_back(c->getGrowth());
    double avg = computeAverage(growths);
    cout << "\n[Template] Average crop growth: " << (avg * 100) << "%\n";
}

void lambdaDemo() {
    cout << "[Lambda] Irrigation advice:\n";
    for (auto& f : fields) {
        auto advice = [](double m) -> string {
            if (m < 30) return "URGENT irrigate";
            if (m < 50) return "Schedule irrigation";
            return "OK";
        };
        cout << "  " << f->getName() << " (" << f->getMoisture() << "%): "
             << advice(f->getMoisture()) << endl;
    }
}
```

```
};

int main() {
    SimulationEngine engine;
    cout << "============================================================\n";
    cout << "CROP SIMULATION - MILESTONE 5 (Concurrency, Enums, Templates, Lambdas)\n";
    cout << "============================================================\n";
    cout << "Commands: s=start, t=step, p=print, h=harvest, a=average, l=lambda, q=quit\n\n";

    char cmd;
    while (true) {
        cout << "> ";
        cin >> cmd;
        switch(cmd) {
            case 's': engine.start(); cout << "Started.\n"; break;
            case 't': engine.step(); cout << "Stepped one day.\n"; break;
            case 'p': engine.showStatus(); break;
            case 'h': engine.harvestAll(); break;
            case 'a': engine.showAverageGrowth(); break;
            case 'l': engine.lambdaDemo(); break;
            case 'q': engine.stop(); cout << "Exiting.\n"; return 0;
            default: cout << "Invalid command.\n";
        }
    }
}
```

```
// milestone6_win_fixed.cpp - No condition_variable, pure Win32 API
#include <windows.h>
#include <commctrl.h>
#include <string>
#include <vector>
#include <memory>
#include <thread>
#include <mutex>
#include <atomic>
#include <cmath>
#include <sstream>
#include <iomanip>
#include <chrono>

using namespace std;

// ------------------------- Enumerations -------------------------
enum class CropStage { SEEDLING, VEGETATIVE, REPRODUCTIVE, MATURING, READY,
HARVESTED };
enum class IrrigationState { NONE, LOW, MODERATE, HIGH, CRITICAL };

string StageStr(CropStage s) {
    switch(s) {
        case CropStage::SEEDLING: return "Seedling";
        case CropStage::VEGETATIVE: return "Vegetative";
```

```
        case CropStage::REPRODUCTIVE: return "Reproductive";
        case CropStage::MATURING: return "Maturing";
        case CropStage::READY: return "Ready";
        case CropStage::HARVESTED: return "Harvested";
        default: return "?";
    }
}

string IrriStr(IrrigationState s) {
    switch(s) {
        case IrrigationState::NONE: return "None";
        case IrrigationState::LOW: return "Low";
        case IrrigationState::MODERATE: return "Moderate";
        case IrrigationState::HIGH: return "High";
        case IrrigationState::CRITICAL: return "Critical";
        default: return "?";
    }
}

// ------------------------- Field (non-copyable) -------------------------
class Field {
    string name;
    double moisture;
    mutable mutex mtx;
public:
    Field(const string& n) : name(n), moisture(50.0) {}
```

```
    Field(const Field&) = delete;
    Field& operator=(const Field&) = delete;

    void update(double rain, double temp) {
        lock_guard<mutex> lock(mtx);
        double evap = 2.0 + temp/15.0;
        moisture += rain*0.8 - evap;
        if (moisture < 0) moisture = 0;
        if (moisture > 100) moisture = 100;
    }
    void irrigate(double amount) {
        lock_guard<mutex> lock(mtx);
        moisture += amount / 100.0;
        if (moisture > 100) moisture = 100;
    }
    double getMoisture() const { lock_guard<mutex> lock(mtx); return moisture; }
    IrrigationState need() const {
        double m = getMoisture();
        if (m >= 70) return IrrigationState::NONE;
        if (m >= 50) return IrrigationState::LOW;
        if (m >= 30) return IrrigationState::MODERATE;
        if (m >= 15) return IrrigationState::HIGH;
        return IrrigationState::CRITICAL;
    }
    string getName() const { return name; }
};
```

```
// ------------------------- Crop (non-copyable) -------------------------
class Crop {
    string name;
    double growth;
    double health;
    double waterReq, tempOpt, maxYield, baseRate;
    bool harvested;
    mutable mutex mtx;
public:
    Crop(const string& n, double wr, double topt, double myield, double br)
        : name(n), growth(0.0), health(1.0), waterReq(wr), tempOpt(topt),
          maxYield(myield), baseRate(br), harvested(false) {}
    Crop(const Crop&) = delete;
    Crop& operator=(const Crop&) = delete;

    void update(double temp, double waterSupply) {
        if (harvested) return;
        double waterStress = (waterSupply <= 0) ? 0 : min(1.0, waterSupply / waterReq);
        double tempStress = (abs(temp - tempOpt) > 20) ? 0 : 1.0 - abs(temp - tempOpt)/20.0;
        double rate = baseRate * waterStress * tempStress;
        rate *= (1.0 - growth * 0.5);
        if (rate < 0) rate = 0;
        if (rate > 0.05) rate = 0.05;
        lock_guard<mutex> lock(mtx);
        growth += rate;
```

```
        if (growth > 1.0) growth = 1.0;
        health = health * 0.99 + (waterStress * tempStress) * 0.01;
    }
    double getGrowth() const { lock_guard<mutex> lock(mtx); return growth; }
    double getHealth() const { lock_guard<mutex> lock(mtx); return health; }
    CropStage stage() const {
        lock_guard<mutex> lock(mtx);
        if (harvested) return CropStage::HARVESTED;
        if (growth >= 0.9) return CropStage::READY;
        if (growth >= 0.75) return CropStage::MATURING;
        if (growth >= 0.5) return CropStage::REPRODUCTIVE;
        if (growth >= 0.25) return CropStage::VEGETATIVE;
        return CropStage::SEEDLING;
    }
    string getName() const { return name; }
    double harvest() {
        lock_guard<mutex> lock(mtx);
        if (harvested || growth < 0.9) return 0;
        double yield = maxYield * growth * health;
        harvested = true;
        return yield;
    }
};

// ------------------------ Simulation Engine (no condition_variable) ------------------------
class Engine {
```

```
vector<unique_ptr<Field>> fields;
vector<unique_ptr<Crop>> crops;
atomic<int> day{0};
atomic<bool> running{false};
atomic<bool> paused{false};
thread simThread;

void step() {
    day++;
    double temp = 20.0 + 5.0 * sin(day * 0.1);
    double rain = (day % 4 == 0) ? 5.0 : 0.0;

    vector<thread> workers;
    for (auto& f : fields) {
        workers.emplace_back([&f, rain, temp]() {
            f->update(rain, temp);
            if (f->need() == IrrigationState::HIGH || f->need() == IrrigationState::CRITICAL)
                f->irrigate(2000);
        });
    }
    double avgMoisture = 50.0;
    if (!fields.empty()) {
        double sum = 0;
        for (auto& f : fields) sum += f->getMoisture();
        avgMoisture = sum / fields.size();
    }
```

```
        for (auto& c : crops) {
            workers.emplace_back([&c, temp, avgMoisture]() {
                c->update(temp, avgMoisture);
            });
        }
        for (auto& t : workers) t.join();
    }

    void runLoop() {
        while (running) {
            if (!paused) {
                step();
            }
            this_thread::sleep_for(chrono::milliseconds(200));
        }
    }

public:
    Engine() {
        fields.push_back(make_unique<Field>("North Field"));
        fields.push_back(make_unique<Field>("South Field"));
        crops.push_back(make_unique<Crop>("Wheat", 4.5, 20.0, 6.0, 0.015));
        crops.push_back(make_unique<Crop>("Corn", 6.0, 25.0, 12.0, 0.018));
        crops.push_back(make_unique<Crop>("Tomato", 3.5, 22.0, 40.0, 0.025));
    }
```

```
    ~Engine() { stop(); }

    void start() { if (!running) { running = true; paused = false; simThread =
thread(&Engine::runLoop, this); } }
    void pause() { paused = true; }
    void resume() { paused = false; }
    void stop() { running = false; if (simThread.joinable()) simThread.join(); }
    void stepOnce() { step(); }

    int getDay() const { return day.load(); }
    const vector<unique_ptr<Field>>& getFields() const { return fields; }
    const vector<unique_ptr<Crop>>& getCrops() const { return crops; }
};

// ------------------------- GUI -------------------------
Engine engine;
HWND hProgressWheat, hProgressCorn, hProgressTomato;
HWND hTextWheat, hTextCorn, hTextTomato, hField1, hField2, hDayEdit;
HINSTANCE hInst;

void UpdateUI() {
    auto& crops = engine.getCrops();
    if (crops.size() >= 3) {
        double g1 = crops[0]->getGrowth();
        double g2 = crops[1]->getGrowth();
        double g3 = crops[2]->getGrowth();
```

```
        SendMessage(hProgressWheat, PBM_SETPOS, (WPARAM)(g1*100), 0);
        SendMessage(hProgressCorn, PBM_SETPOS, (WPARAM)(g2*100), 0);
        SendMessage(hProgressTomato, PBM_SETPOS, (WPARAM)(g3*100), 0);

        string s1 = crops[0]->getName() + ": " + StageStr(crops[0]->stage());
        string s2 = crops[1]->getName() + ": " + StageStr(crops[1]->stage());
        string s3 = crops[2]->getName() + ": " + StageStr(crops[2]->stage());
        SetWindowText(hTextWheat, s1.c_str());
        SetWindowText(hTextCorn, s2.c_str());
        SetWindowText(hTextTomato, s3.c_str());
    }
    auto& fields = engine.getFields();
    if (fields.size() >= 2) {
        stringstream ss1, ss2;
        ss1 << fields[0]->getName() << ": " << fixed << setprecision(1) << fields[0]->getMoisture()
            << "%  Need: " << IrriStr(fields[0]->need());
        ss2 << fields[1]->getName() << ": " << fixed << setprecision(1) << fields[1]->getMoisture()
            << "%  Need: " << IrriStr(fields[1]->need());
        SetWindowText(hField1, ss1.str().c_str());
        SetWindowText(hField2, ss2.str().c_str());
    }
    char buf[32];
    sprintf(buf, "Day: %d", engine.getDay());
    SetWindowText(hDayEdit, buf);
}
```

```
void CALLBACK TimerProc(HWND, UINT, UINT_PTR, DWORD) {
    UpdateUI();
}

void OnStart() { engine.start(); }
void OnPause() { engine.pause(); }
void OnStop() { engine.stop(); UpdateUI(); }
void OnStep() { engine.stepOnce(); UpdateUI(); }
void OnReset() {
    engine.stop();
    engine = Engine();  // fresh engine (move assignment should be okay if we implement)
    UpdateUI();
}
void OnHarvest() {
    string msg;
    for (auto& c : engine.getCrops()) {
        double y = c->harvest();
        if (y > 0) msg += "Harvested " + c->getName() + ": " + to_string(y) + " t/ha\n";
    }
    if (msg.empty()) msg = "No crops ready.";
    MessageBox(NULL, msg.c_str(), "Harvest", MB_OK);
    UpdateUI();
}

LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
    switch(msg) {
```

```
    case WM_CREATE:

        CreateWindow(WC_STATIC, "Crop Growth", WS_CHILD | WS_VISIBLE, 20, 20, 100, 20, hwnd, NULL, hInst, NULL);

        hProgressWheat = CreateWindow(PROGRESS_CLASS, NULL, WS_CHILD | WS_VISIBLE, 130, 20, 300, 20, hwnd, NULL, hInst, NULL);

        hTextWheat = CreateWindow(WC_STATIC, "Wheat", WS_CHILD | WS_VISIBLE, 440, 20, 150, 20, hwnd, NULL, hInst, NULL);


        CreateWindow(WC_STATIC, "", WS_CHILD | WS_VISIBLE, 20, 50, 100, 20, hwnd, NULL, hInst, NULL);

        hProgressCorn = CreateWindow(PROGRESS_CLASS, NULL, WS_CHILD | WS_VISIBLE, 130, 50, 300, 20, hwnd, NULL, hInst, NULL);

        hTextCorn = CreateWindow(WC_STATIC, "Corn", WS_CHILD | WS_VISIBLE, 440, 50, 150, 20, hwnd, NULL, hInst, NULL);


        CreateWindow(WC_STATIC, "", WS_CHILD | WS_VISIBLE, 20, 80, 100, 20, hwnd, NULL, hInst, NULL);

        hProgressTomato = CreateWindow(PROGRESS_CLASS, NULL, WS_CHILD | WS_VISIBLE, 130, 80, 300, 20, hwnd, NULL, hInst, NULL);

        hTextTomato = CreateWindow(WC_STATIC, "Tomato", WS_CHILD | WS_VISIBLE, 440, 80, 150, 20, hwnd, NULL, hInst, NULL);


        CreateWindow(WC_STATIC, "Fields:", WS_CHILD | WS_VISIBLE, 20, 120, 100, 20, hwnd, NULL, hInst, NULL);

        hField1 = CreateWindow(WC_STATIC, "", WS_CHILD | WS_VISIBLE, 20, 140, 400, 20, hwnd, NULL, hInst, NULL);

        hField2 = CreateWindow(WC_STATIC, "", WS_CHILD | WS_VISIBLE, 20, 160, 400, 20, hwnd, NULL, hInst, NULL);
```

```
        hDayEdit = CreateWindow(WC_STATIC, "Day: 0", WS_CHILD | WS_VISIBLE, 20, 200, 100, 20, hwnd, NULL, hInst, NULL);


        CreateWindow(WC_BUTTON, "Start", WS_CHILD | WS_VISIBLE, 20, 240, 80, 30, hwnd, (HMENU)1, hInst, NULL);
        CreateWindow(WC_BUTTON, "Pause", WS_CHILD | WS_VISIBLE, 110, 240, 80, 30, hwnd, (HMENU)2, hInst, NULL);
        CreateWindow(WC_BUTTON, "Stop",  WS_CHILD | WS_VISIBLE, 200, 240, 80, 30, hwnd, (HMENU)3, hInst, NULL);
        CreateWindow(WC_BUTTON, "Step",  WS_CHILD | WS_VISIBLE, 290, 240, 80, 30, hwnd, (HMENU)4, hInst, NULL);
        CreateWindow(WC_BUTTON, "Reset", WS_CHILD | WS_VISIBLE, 380, 240, 80, 30, hwnd, (HMENU)5, hInst, NULL);
        CreateWindow(WC_BUTTON, "Harvest",WS_CHILD | WS_VISIBLE, 470, 240, 80, 30, hwnd, (HMENU)6, hInst, NULL);


        SendMessage(hProgressWheat, PBM_SETRANGE, 0, MAKELPARAM(0, 100));
        SendMessage(hProgressCorn, PBM_SETRANGE, 0, MAKELPARAM(0, 100));
        SendMessage(hProgressTomato, PBM_SETRANGE, 0, MAKELPARAM(0, 100));


        UpdateUI();
        SetTimer(hwnd, 1, 500, TimerProc);
        break;
    case WM_COMMAND:
        switch(LOWORD(wParam)) {
            case 1: OnStart(); break;
            case 2: OnPause(); break;
            case 3: OnStop(); break;
```

```
            case 4: OnStep(); break;
            case 5: OnReset(); break;
            case 6: OnHarvest(); break;
        }
        break;
    case WM_DESTROY:
        KillTimer(hwnd, 1);
        engine.stop();
        PostQuitMessage(0);
        break;
    default:
        return DefWindowProc(hwnd, msg, wParam, lParam);
  }
  return 0;
}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE, LPSTR, int nCmdShow) {
  hInst = hInstance;
  INITCOMMONCONTROLSEX icc = {sizeof(icc), ICC_PROGRESS_CLASS};
  InitCommonControlsEx(&icc);

  WNDCLASS wc = {};
  wc.lpfnWndProc = WndProc;
  wc.hInstance = hInstance;
  wc.hCursor = LoadCursor(NULL, IDC_ARROW);
  wc.hbrBackground = (HBRUSH)(COLOR_WINDOW+1);
```

```
    wc.lpszClassName = "CropSimClass";
    RegisterClass(&wc);

    HWND hwnd = CreateWindowEx(0, "CropSimClass", "Crop Growth Simulation - Milestone 6",
                               WS_OVERLAPPEDWINDOW & ~WS_MAXIMIZEBOX & ~WS_THICKFRAME,
                               CW_USEDEFAULT, CW_USEDEFAULT, 600, 350,
                               NULL, NULL, hInstance, NULL);
    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);

    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    return msg.wParam;
}
```